LẬP TRÌNH WEB

CHƯƠNG 4

LÀM VIỆC VỚI FORM

Giảng viên: TS. Đặng Trung Thành

Chuẩn đầu ra của chương

- Lý thuyết
  - Giải thích được vai trò và chức năng của Form trong ứng dụng Web
  - Giải thích được các tính năng của thư viện Django Forms
  - Giải thích được ý nghĩa của việc kiểm tra tính hợp lệ và làm sạch dữ liệu từ Form
  - Giải thích được các thư viện hỗ trợ kiểm tra tính hợp lệ dữ liệu của Django
- Thực hành
  - Tạo được form với các thành phần giao diện theo yêu cầu
  - Kiểm tra được tính hợp lệ và truy xuất được dữ liệu từ các thành phần của Form
  - Đặt được giá trị khởi tạo và làm sạch dữ liệu trên form
  - Liên kết được form với Model của Django

Nội dung

- Cơ bản về Form
- Thư viện Forms của Django
- Kiểm tra tính hợp lệ của dữ liệu nhập

4.1 Cơ bản về Form

Form là gì

- Form chứa các thành phần nhập liệu để gửi đến trình chủ web
  - Các thành phần nhập liệu có nhiều dạng khác nhau: hộp nhập văn bản (textbox), hộp danh sách sổ xuống (dropdown listbox), hộp kiểm (checkbox)...
  - Dữ liệu được gửi đi dưới dạng các cặp ***khóa – giá trị*** *(key – value)*
    + Khóa được xác định thông qua thuộc tính **name** của thành phần nhập liệu
    + Giá trị tương ứng với một khóa có thể là một giá trị đơn lẻ hoặc một tập giá trị (như tập giá trị từ các hộp kiểm)

Ví dụ: *Trang Đăng nhập chứa 2 hộp nhập văn bản để người dùng nhập vào username và password, và một nút gửi (submit button) thường có nhãn là Login/ Sign in để gửi dữ liệu đến trình chủ web*

4.1 Cơ bản về Form

Form là gì

- Được định nghĩa bởi cặp tag **<form> ... </form>**
- Các thuộc tính chính của Form:
  - **method**:
    + Phương thức để gửi dữ liệu của form đến trình chủ web, là **get** hoặc **post**.
    + Mặc định là phương thức **get**.
  - **action**:
    + URL (hoặc đường dẫn) đến máy chủ để gửi dữ liệu đến.
    + Mặc định là địa chỉ hiện tại của trang web chứa form.
  - **enctype**:
    + Loại mã hóa dữ liệu của form.
    + Mặc định là **application/x-www-form-urlencoded**.
    + Khi gửi tập tin, giá trị cần đặt là **multipart/form-data**

4.1 Cơ bản về Form

Form là gì

Ví dụ:

```
<form method="post" action="/form-submit" enctype="multipart/form-data">
    <!- Đặt các thành phần nhập liệu tại đây-->
</form>
```

- Phương thức gửi dữ liệu đến máy chủ **GET** và **POST**
  - Phương thức GET: Dữ liệu được đưa vào URL

```
GET /form-submit?first_name=Joe&last_name=Bloggs HTTP/1.1
Host: www.example.com
```

  - Phương thức POST: Dữ liệu được gửi ở nội dung gói tin Request

```
POST /form-submit HTTP/1.1
Host: www.example.com
Content-Length: 31
Content-Type: application/x-www-form-urlencoded
first_name=Joe&last_name=Bloggs
```

4.1 Cơ bản về Form

Các loại thành phần nhập liệu trong Form

- Hộp nhập văn bản (Textbox)
  - Văn bản thông thường (text)

```
<input id="id_text_input" type="text‟ name="text_input" value="" placeholder="Enter some text">
```

  - Mật khẩu (password)

```
<input id="id_password_input" type="password‟ name="password_input" value="" placeholder="Your password">
```

  - Số (number)

```
<input id="id_number_input" type="number‟ name="number_input" value="" step="any‟ placeholder="A number">
```

  - Email

```
<input id="id_email_input" type="email‟ name="email_input" value="" placeholder="Your email address">
```

  - Ngày tháng

```
<input id="id_date_input" type="date‟ name="date_input">
```

4.1 Cơ bản về Form

Các loại thành phần nhập liệu trong Form

- Hộp nhập văn bản dài (Textarea)

```
<textarea name="text_area" id="id_text_area‟ placeholder="Enter multiple
lines of text">

</textarea>
```

- Hộp kiểm (Checkbox)

```
<input id="id_checkbox_input" type="checkbox‟ name="checkbox_on"
value="Checkbox Checked‟ checked>
```

- Nút lựa chọn (Radio button)

```
<input id="id_radio_one_input" type="radio" name="radio_input"
value="Value One">
<input id="id_radio_two_input" type="radio‟ name="radio_input"
value="Value Two" checked>
```

4.1 Cơ bản về Form

Các loại thành phần nhập liệu trong Form

- Hộp danh sách (Listbox)
  - Danh sách đơn lựa chọn (single select listbox)

```
<select id="id_favorite_book" name="favorite_book">
    <option value="1">Deep Learning with Keras</option>
    <option value="2">Web Development with Django</option>
</select>
```

  - Danh sách nhiều lựa chọn (multiple select listbox)

```
<select id="id_books_you_own" name="books_you_own" multiple>
    <optgroup label="Non-Fiction">
        <option value="1">Deep Learning with Keras</option>
        <option value="2">Web Development with Django</option>
    </optgroup>
    <optgroup label="Fiction">
        <option value="3">Brave New World</option>
        <option value="4">The Great Gatsby</option>
    </optgroup>
</select>
```

Các loại thành phần nhập liệu trong Form

- Hộp nhập ẩn (Hidden)

```
<input type="hidden" name="hidden_input" value="Hidden Value">
```

- Nhãn (Label)

```
<label for="id_number_input">Number Input</label>
```

- Nút bấm

```
<input type="submit" name="submit_input" value="Submit Input">
```

Hoặc:

```
<button type="submit" name="button_element" value="Button
Element">Button With<strong> Styled</strong> Text </button>
```

  - Type có thể là:
    + **`Submit`**: Nút gửi dữ liệu của form lên máy chủ web
    + **Reset**: Nút xóa nội dung của các thành phần nhập liệu trong form

4.1 Cơ bản về Form

Bảo mật dữ liệu gửi từ Form

- Django có hỗ trợ bảo vệ ứng dụng khỏi bị tấn công CSRF (Cross-Site Request Forger)
- Tấn công CSRF:
  - Khai thác đặc điểm một form có thể gửi dữ liệu đến bất cứ máy chủ web nào (qua thuộc tính **action**)
  - Kẻ tấn công có thể sao chép form của một website và điều chỉnh nội dung rồi dùng form này để gửi nội dung giả mạo đến website
- Cách phòng chống tấn công CSRF của Django:
  - Với mỗi phiên truy cập của người dùng, Django sinh ra một token CSRF và lưu trong một hộp nhập ẩn của form
  - Trình chủ web sẽ so khớp giá trị token CSRF khi có một form gửi dữ liệu về và chỉ chấp nhận các form có giá trị token khớp với giá trị lưu trên trình chủ.

(Sinh viên đọc thêm ở trang 273-275 của Giáo trình)

4.2 Thư viện Forms của Django

Giới thiệu

- Thư viện Forms của Django cho phép tạo nhanh form sử dụng lớp trên Python
  - Bằng cách tạo lớp con kế thừa từ lớp Form (tương tự như cách làm với các lớp Model)
  - Form có thể tự động sinh ra từ lớp đã định nghĩa (trên Python)
- Hỗ trợ kiểm tra tính hợp lệ của dữ liệu nhập
  - Đặc biệt với kiểm tra dữ liệu liên trường (cross-field validation)
- Có thể tự động sinh ra Form từ Model

4.2 Thư viện Forms của Django

Định nghĩa một Form

- Tạo lớp kế thừa từ lớp **django.forms.Form**
- Các thuộc tính của lớp là thể hiện của các lớp con của lớp **django.forms.Field**
- Một số lớp Field được định nghĩa sẵn:
  - **CharField**: Hộp nhập văn bản
  - **EmailField**: Hộp nhập email
  - **BooleanField, IntegerField, FloatField & DecimalField**: Hộp nhập dữ liệu kiểu Boolean, số nguyên, số thực và số thập phân
  - **DateField, TimeField & DateTimeField**: Hộp nhập giá trị ngày tháng và thời gian
  - **ChoiceField & MultipleChoiceField**: Hộp danh sách đơn và đa lựa chọn
  - **FileField**: Hộp chọn để tải tập tin lên máy chủ web
- Tương ứng với mỗi thuộc tính sẽ sinh ra hộp nhập tương ứng (cùng tên với tên thuộc tính)

(Xem thêm về danh sách các lớp con của lớp django.forms.Field tại đây https://docs.djangoproject.com/en/4.0/ref/forms/fields/#built-in-field-classes)

4.2 Thư viện Forms của Django

Định nghĩa một Form

- Có thể thay đổi cách thức hiển thị các trường bằng cách sử dụng widget

  Ví dụ: `password_input = forms.CharField(widget=forms.PasswordInput)`

- Một số lớp Widget được định nghĩa sẵn:
  - Dùng cho hộp nhập văn bản: TextInput, NumberInput, PasswordInput, EmailInput, DateInput, Textarea, ...
  - Dùng cho hộp danh sách lựa chọn: CheckboxInput, Select, SelectMultiple, RadioSelect, CheckboxSelectMultiple
  - Dùng cho tải tập tin lên máy chủ web: FileInput, ClearableFileInput
  - Dùng cho kết hợp nhiều hộp nhập: MultipleHiddenInput, SplitDateTimeWidget, SelectDateWidget,...

  (Xem thêm danh sách các lớp Widget tại đây: https://docs.djangoproject.com/en/4.0/ref/forms/widgets/#splitdatetimewidget)

4.2 Thư viện Forms của Django

Sử dụng Template cho Form

- Để render form trong một template chỉ cần khởi tạo đối tượng form và truyền vào hàm render()

  Ví dụ:

```
def view_function(request):
    form = ExampleForm()
    return render(request, "template.html", {"form": form})
```

- Có thể chọn 1 trong 3 bố cục:
  - **as_table**: Dạng bảng
  - **as_ul**: Dạng danh sách
  - **as_p**: Dạng đoạn văn bản (từng dòng)

  Ví dụ:

```
<form method="post">
    <table>
        {{ form.as_table }}
    </table>
</form>
```

4.3 Kiểm tra tính hợp lệ của dữ liệu nhập

Cơ bản về kiểm tra tính hợp lệ dữ liệu nhập với thư viện Forms

- Một Form trên Django có thể có ràng buộc (**bound**) hoặc không ràng buộc (unbound) với dữ liệu gửi đến.
- Có thể tạo một đối tượng form **có ràng buộc** (ví dụ với dữ liệu gửi từ máy khách **request.POST**) như sau:

```
form = ExampleForm(request.POST)
```

- Form có ràng buộc cho phép kiểm tra tính hợp lệ với các công cụ định nghĩa sẵn:
  - Phương thức **is_valid**: kiểm tra dữ liệu trong form có hợp lệ không
  - Thuộc tính **cleaned_data**: chứa dữ liệu đã chuyển đổi thành dạng phù hợp trên Python (chỉ khả dụng sau khi dữ liệu đã được làm sạch với phương thức **is_valid()**)
- Form của Django cũng hỗ trợ kiểm tra tính hợp lệ của dữ liệu nhập ngay trên máy khách (trình duyệt web) cho nhiều loại hộp nhập

4.3 Kiểm tra tính hợp lệ của dữ liệu nhập

Một số bộ kiểm tra tính hợp lệ dữ liệu nhập được định nghĩa sẵn

- **required**: mặc định là True. Yêu cầu phải nhập dữ liệu cho hộp nhập
- **max_length**: đặt số ký tự tối đa có thể nhập (cho các hộp nhập văn bản)
- **min_length**: đặt số ký tự tối thiểu cần phải nhập (cho các hộp nhập văn bản)
- **max_value**: giá trị tối đa cho phép (cho các hộp nhập giá trị kiểu số)
- **min_value**: giá trị tối thiểu cho phép (cho các hộp nhập giá trị kiểu số
- **max_digits**: số chữ số tối đa, tính cả số ở trước và sau dấu chấm thập phân (dùng cho **DecimalField**)
- **decimal_places**: số chữ số tối đa ở phần lẻ của số thập phân

4.3 Kiểm tra tính hợp lệ của dữ liệu nhập

Kiểm tra và làm sạch dữ liệu tùy chỉnh

- Có thể kiểm tra tính hợp lệ, xử lý, chuyển đổi dữ liệu gửi từ form bằng cách định nghĩa các phương thức thuộc lớp Form
- Kiểm tra tính hợp lệ tùy chỉnh:
  - Thực hiện bằng các **validator**: Là một hàm nhận vào giá trị và trả về ngoại lệ **`django.core.exceptions.ValidationError`** nếu dữ liệu không hợp lệ

    Ví dụ:

    ```
    from django.core.exceptions import ValidationError
    def validate_lowercase(value):
        if value.lower() != value:
            raise ValidationError("{} is not lowercase.".format(value))
    ```

  - Validator áp dụng được cho *nhiều trường/ hộp nhập* đồng thời qua thuộc tính **validators**

    Ví dụ:

    ```
    class ExampleForm(forms.Form):
        text_input1 = forms.CharField(validators=[validate_lowercase])
        text_input2 = forms.CharField(validators=[validate_lowercase]
    ```

4.3 Kiểm tra tính hợp lệ của dữ liệu nhập

Kiểm tra và làm sạch dữ liệu tùy chỉnh

- Xử lý và chuyển đổi dữ liệu với phương thức **clean()**
  - Kiểm tra cho từng trường, tên phương thức theo quy tắc: **clean_<Tên_trường>**
    - **<Tên_Trường>:** Tên của đối tượng hộp nhập (trên **python**)
  - Cho phép kiểm tra tính hợp lệ của dữ liệu nhập và/hoặc làm sạch dữ liệu

Ví dụ 1:

```
class ExampleForm(forms.Form):
    text_input = forms.CharField()
    …
    def clean_text_input(self):
        value = self.cleaned_data["text_input"]
        if value.lower() != value:
            raise ValidationError("{} is not lowercase.".format(value))
        return value
```

→ *Kiểm tra tính hợp lệ của dữ liệu nhập (dữ liệu nhập có phải là toàn chữ thường không)*

4.3 Kiểm tra tính hợp lệ của dữ liệu nhập

Kiểm tra và làm sạch dữ liệu tùy chỉnh

- Xử lý và chuyển đổi dữ liệu với phương thức **clean()**

Ví dụ 2:

```
class ExampleForm(forms.Form):
    text_input = forms.CharField()
    …
    def clean_text_input(self):
        value = self.cleaned_data['text_input’]
        return value.lower()
```

→ *Chỉ làm sạch dữ liệu nhập (chuyển tất cả thành chữ thường)*

4.3 Kiểm tra tính hợp lệ của dữ liệu nhập

Kiểm tra và làm sạch dữ liệu tùy chỉnh

- Xử lý và chuyển đổi dữ liệu với phương thức **clean()**
  - Cho phép kiểm tra ràng buộc trên *nhiều trường/ hộp nhập*

Ví dụ: *Người dùng chỉ bắt buộc phải nhập email nếu họ có chọn mục đăng ký tài khoản*

```
class NewsletterSignupForm(forms.Form):
    signup = forms.BooleanField(label="Sign up to
newsletter?",required=False)
    email = forms.EmailField(help_text="Your email address to
subscribe", required=False)
    def clean(self):
        cleaned_data = super().clean()
        if cleaned_data["signup"] and not cleaned_data.get("email"):
            self.add_error("email", "Your email address is required if
signing up for the newsletter.")
```

4.3 Kiểm tra tính hợp lệ của dữ liệu nhập

Thêm chỉ dẫn nhập và giá trị khởi tạo

- Thêm chỉ dẫn nhập (placeholder)
  - Sử dụng **widget**

    Ví dụ:

```
class ExampleForm(forms.Form):
    text_field = forms.CharField(widget=forms.TextInput(
attrs={"placeholder": "Text Placeholder"})
)
```

- Thêm giá trị khởi tạo
  - Sử dụng thuộc tính **`initial`** cho từng trường

    Ví dụ:

```
text_field = forms.CharField(initial="Initial Value", …)
```

  - Sử dụng thuộc tính **`initial`** của form

    Ví dụ:

```
initial = {"text_field": "Text Value", "email_field": "user@example.com"}
form = ExampleForm(initial=initial)
```

4.3 Kiểm tra tính hợp lệ của dữ liệu nhập

Nhập dữ liệu cho Model với lớp ModelForm

- Cho phép tạo form nhập liệu trực tiếp cho các Model
  - Kế thừa các quy tắc kiểm tra tính hợp lệ từ định nghĩa của Model
  - Có thể khởi tạo giá trị mặc định cho các trường nhập qua tham số **__init__** (biến **instance**)
  - Có phương thức **save()** để lưu dữ liệu vào CSDL

Ví dụ:

```
from .models import Publisher
class PublisherForm(forms.ModelForm):
    class Meta:
        model = Publisher
        fields = ("name", "website", "email")
```

  - **Lưu ý:** có thể dùng biến __all__ để đại diện cho tất cả cả trường hoặc dùng thuộc tính exclude để loại trừ các trường khỏi form nhập.

Hết chương 4